Reverso for QUIC

draft-frochet-quicwg-reverso-for-quic-01

Florentin Rochet

University of Namur, UNamur

2026-07-22

UNIVERSITÉ DE NAMUR

Paper TL;DR

QUIC provides a secure contiguous ordered multistream abstraction

- We cannot write a QUIC v1 library with contiguous zero-copy on the receiver side.
  - Encrypted data potentially surrounded by encrypted control is to blame.
- A new QUIC version?

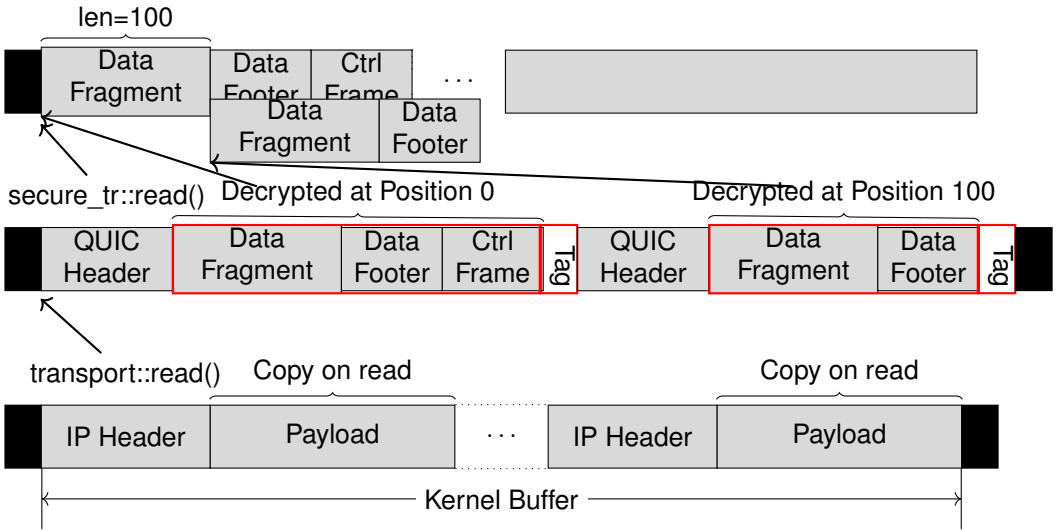
Details on paper “Contiguous Zero-Copy for Encrypted Transport Protocols”, published in ACM SIGCOMM CCR, Jan. 2026.

What changes from QUIC v1?

1. A few more bytes in the short header. No change to the header protection algorithm.
   - Worst case: 5 bytes in QUIC v1, 13 bytes in vReverso.
   - Two more "compressed" variable integers: a stream id, and an offset.
2. A stream frame (if any sent) must be the first frame within the packet. Multiple stream frames are allowed.
3. The ordering of frame elements are **reversed** on the wire, all frames.

These 3 modifications permit implementing “contiguous zero-copy” in expectation.

How? Drive data reassembly using decryption's hidden copy



Why reversing frame ordering? (other solutions are possible)

- Minimizes added info in the short header (no need to locate where the control begins).
- QUIC v1 and vReverso do not need to handle writing/reading differently on/from the wire, assuming a buffer abstraction processing or writing bytes both directions exists.

both versions can be maintained with the same code.

One possible abstraction (protocol independent)

```rust
struct Frame {
    ty: u8,
    id: u16,
    thing: VarInt,
}
impl Frame {
    /// This code is oblivious to whether buf is forward or
    backward.
    /// It reads the "next" fields in the defined logical order.
    fn parse<R: OctetsRead>(buf: &mut R) -> Result<Self> {
        let ty = buf.get_u8()?;
        let id = buf.get_u16()?;
        let thing = buf.get_varint()?;
        Ok(Frame { ty, id, thing })
    }
}
```

Benefits? Drawbacks?

- Benefits:
  - Better code efficiency leads to higher throughput in some scenarios.
  - HTTP/3 can expose a whole data frame of arbitrary size in zero-copy.
  - Applies to derived protocols, e.g., QMux.
  - Standardizing contiguous zero-copy may help avoiding eclectic solutions.
- Drawbacks:
  - Maintenance cost of v1 and vReverso is unclear (hopefully limited).
  - Some edge cases require in-place decryption then copy (e.g., some stream data overlap; spilling across chunk boundary, unordered packets), similarly to QUIC v1.

Backup slides

Packet header details

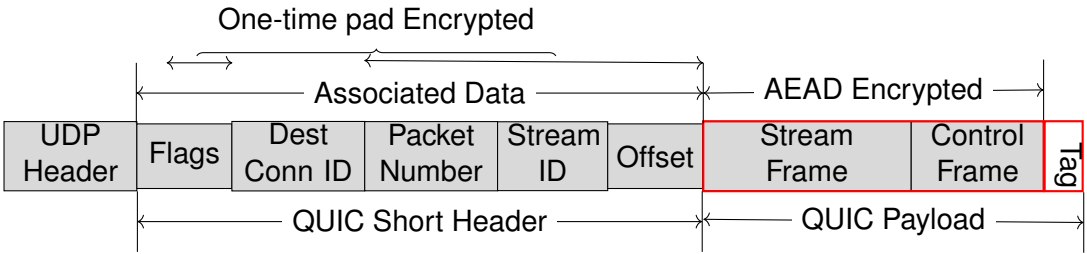


Figure: QUIC vReverso short header packet, exchanged after the session is established

- Stream ID: using same encoding/decoding than packet numbers?
- Offset: same encoding/decoding than packet numbers is currently used.

async-quiceh echo-reply API example

```rust
async fn echo_stream(
    mut send: SendStream, mut recv: RecvStream,
) -> anyhow::Result<()> {
    while !recv.fin() {
        // Recv a whole chunk in zero-copy (64 KiB by default) if
        // vReverso was negotiated. Otherwise copies happen within
        // the library for QUIC v1.
        let (chunk, fin) = recv.stream_recv().await?;
        // Wrapper type to send in zero-copy.
        let buf = ServerBuf::from(chunk);
        let written = send.write_chunk(buf, fin).await?;
        info!("Echoed {written} bytes on stream {}", send.id());
    }
    Ok(())
}
```

draft-banks-quic-performance-00

Measurements: Download 2GiB per stream in a loop during 60 seconds, Network: Loopback, CPU: AMD Ryzen 9 8940HX, kernel 7.0.0-28-generic

Clients		Servers			
		async-quiceh		Quinn	picoquic
		vReverso	v1		
async-quiceh	vReverso	15,4 Gb/s	N/A	N/A	N/A
	v1	N/A	15,2 Gb/s	14,4 Gb/s	15,2 Gb/s
Quinn		N/A	15,3 Gb/s	14,5 Gb/s	14,9 Gb/s
picoquic		N/A	4,6 Gb/s	4,77 Gb/s	4,53 Gb/s

Numbers are to be taken with a grain of salt. This is a very incomplete view of what is happening. Note: had some issues with MsQuic on linux and results were not included.